

1. Launch one aws ec2 and install jenkins.

1. **Login to AWS Console**
2. Go to **EC2** → **Launch Instance**
3. Select **Ubuntu 24.04 LTS**
4. Select **Instance Type – t2.micro**
5. Create/select a **Key Pair**
6. Configure **Security Group**
 - SSH – Port **22**
 - Custom TCP – Port **8080**
7. **Launch EC2 Instance**
8. Connect to EC2 using **SSH**
9. Update Ubuntu:

```
sudo apt update
```

10. Install Java:

```
sudo apt install fontconfig openjdk-21-jre -y
```

11. Add the **Jenkins repository**
12. Update package list:

```
sudo apt update
```

13. Install Jenkins:

```
sudo apt install jenkins -y
```

14. Start Jenkins:

```
sudo systemctl start jenkins
```

15. Check Jenkins status:

```
sudo systemctl status jenkins
```

16. Enable Jenkins:

```
sudo systemctl enable jenkins
```

17. Get the Jenkins initial password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

18. Open browser:

```
http://<EC2-Public-IP>:8080
```

19. Enter the **initial password**
20. Select **Install suggested plugins**

21. Create Jenkins Admin User
22. Jenkins Dashboard is ready ✓

The screenshot shows the AWS Management Console interface for EC2 Instances. The instance 'i-02ffff435ebc804be' is listed with a status of 'Running'. A tooltip indicates that the public IPv4 address '51.20.181.27' has been copied. The instance details show it is a t3.micro instance in the eu-north-1a availability zone.

The screenshot shows the AWS Management Console interface for Security Groups. The inbound rules for the security group 'sg-0565ebef96d035ba0' are displayed. The rules include SSH (port 22), HTTP (port 80), RDP (port 3389), and Custom TCP (port 8082). The source for all rules is '0.0.0.0/0'.

```
Vineeth@LAPTOP-LMH2T0GS MINGW64 ~/Downloads (master)
$ chmod 400 "cnode-server.pem"

Vineeth@LAPTOP-LMH2T0GS MINGW64 ~/Downloads (master)
$ ssh -i "cnode-server.pem" ubuntu@ec2-51-20-181-27.eu-north-1.compute.amazonaws.com
welcome to ubuntu 26.04 LTS (GNU/Linux 7.0.0-1006-aws x86_64)
```

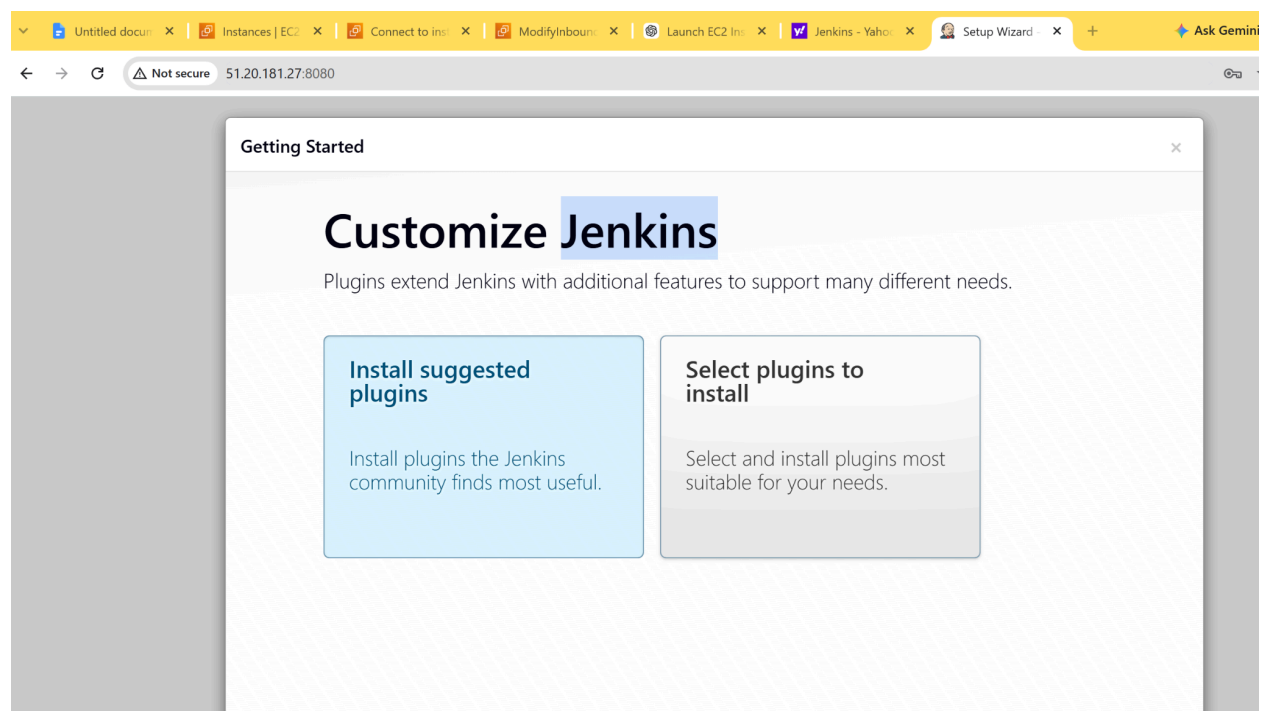
```
ubuntu@ip-172-31-17-82:~$ sudo apt upgrade -y
The following package was automatically installed and is no longer required:
  pollinate
Use 'sudo apt autoremove' to remove it.
```

```

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-17-82:~$ sudo systemctl start jenkins
ubuntu@ip-172-31-17-82:~$ sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-09-17 07:25:19 UTC; 28s ago
     Invocation: 61131230e5b7492d9e77f7b7621dd3f3
   Main PID: 29680 (java)
     Tasks: 41 (limit: 627)
    Memory: 317.7M (peak: 335.9M)
       CPU: 21.455s
    CGroup: /system.slice/jenkins.service
            └─29680 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenki

Sep 17 07:25:13 ip-172-31-17-82 jenkins[29680]: [LF]>
Sep 17 07:25:13 ip-172-31-17-82 jenkins[29680]: [LF]> *****
Sep 17 07:25:13 ip-172-31-17-82 jenkins[29680]: [LF]> *****
Sep 17 07:25:13 ip-172-31-17-82 jenkins[29680]: [LF]> *****
Sep 17 07:25:19 ip-172-31-17-82 jenkins[29680]: 2026-09-17 07:25:19.375+0000 [id=32]
Sep 17 07:25:19 ip-172-31-17-82 jenkins[29680]: 2026-09-17 07:25:19.413+0000 [id=24]
Sep 17 07:25:19 ip-172-31-17-82 systemd[1]: Started jenkins.service - Jenkins Continuo
Sep 17 07:25:20 ip-172-31-17-82 jenkins[29680]: 2026-09-17 07:25:20.118+0000 [id=50]
Sep 17 07:25:20 ip-172-31-17-82 jenkins[29680]: 2026-09-17 07:25:20.120+0000 [id=50]
Sep 17 07:25:24 ip-172-31-17-82 jenkins[29680]: 2026-09-17 07:25:24.484+0000 [id=69]
ubuntu@ip-172-31-17-82:~$ sudo systemctl enable jenkins
Synchronizing state of jenkins.service with SysV service script with /usr/lib/systemd/
systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable jenkins
ubuntu@ip-172-31-17-82:~$ sudo cat /var/lib/jenkins/secrets/initialAdminPassword
fc0f06d060154c9aadff5b183b3c5117

```



2. Setup Apache Tomcat on Ubuntu Virtual Machine

1. Connect to Ubuntu VM

Connect to the Ubuntu server using SSH.

```
ssh -i "key.pem" ubuntu@<Public-IP>
```

2. Update the system

```
sudo apt update
```

3. Install Java

Tomcat requires Java to run.

```
sudo apt install openjdk-17-jdk -y
```

Check Java:

```
java -version
```

4. Create Tomcat directory

```
sudo mkdir /opt/tomcat
```

5. Download Apache Tomcat

```
cd /tmp
```

Download the required Tomcat version:

```
wget <Tomcat-download-link>
```

6. Extract Tomcat

```
sudo tar -xvzf apache-tomcat-<version>.tar.gz -C /opt/tomcat --strip-components=1
```

7. Give execute permission

```
sudo chmod +x /opt/tomcat/bin/*.sh
```

8. Start Tomcat

```
sudo /opt/tomcat/bin/startup.sh
```

Expected output:

Tomcat started.

9. Check Tomcat process

```
ps -ef | grep tomcat
```

10. Check Tomcat port

Tomcat uses **port 8080** by default.

```
sudo ss -ltnp | grep 8080
```

Expected:

```
LISTEN ... 0.0.0.0:8080
```

11. Test Tomcat locally

```
curl http://localhost:8080
```

If HTML output is displayed, Tomcat is working.

12. Allow port 8080 in AWS Security Group

Go to:

EC2 → **Security Groups** → **Inbound rules** → **Edit inbound rules**

Add:

Type: Custom TCP

Port: 8080

Source: 0.0.0.0/0

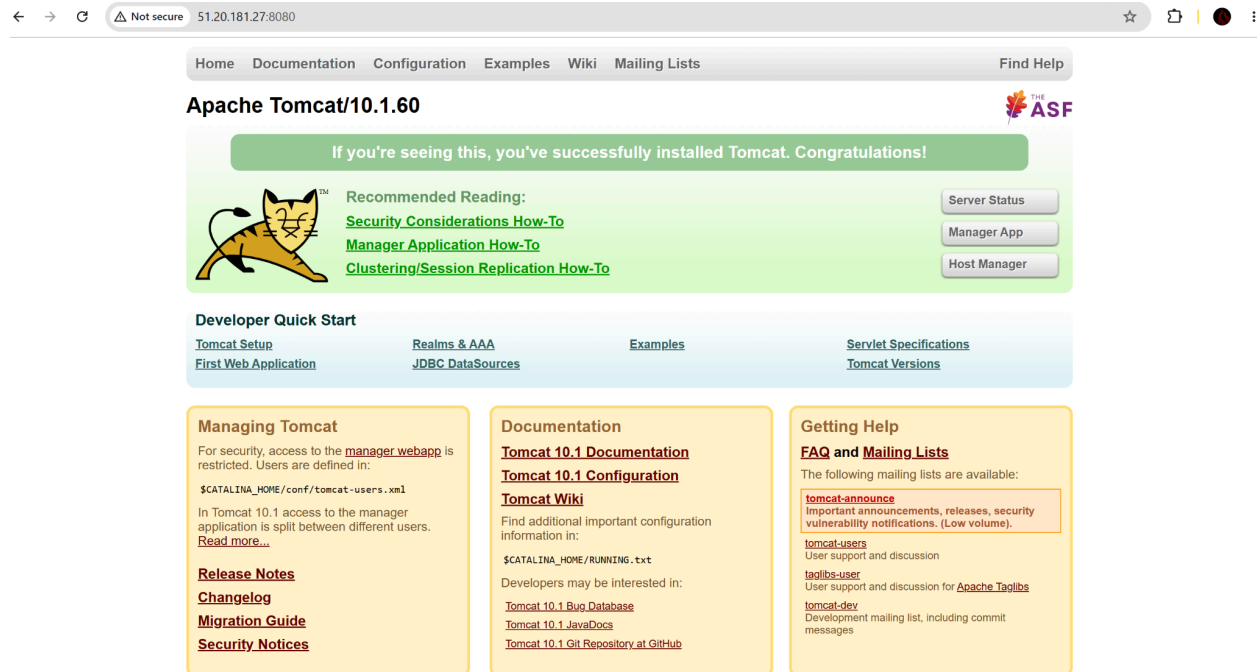
Save the rule.

13. Access Tomcat from browser

Open:

```
http://<Public-IP>:8080
```

You should see the **Apache Tomcat welcome page**.



3. Setup Docker Desktop on your local windows.

4. **Enable WSL 2**

wsl --install

Restart the computer.

2. **Check WSL**

wsl --status

3. **Download Docker Desktop**

[Docker Desktop Official Download](#)

4. **Install Docker Desktop**

- Open the installer.
 - Select **Use WSL 2 based engine**.
 - Complete installation.
 - Restart if required.
5. **Open Docker Desktop** and wait until it shows **Running**.
6. **Verify Docker**
Open a **new PowerShell**:

docker --version

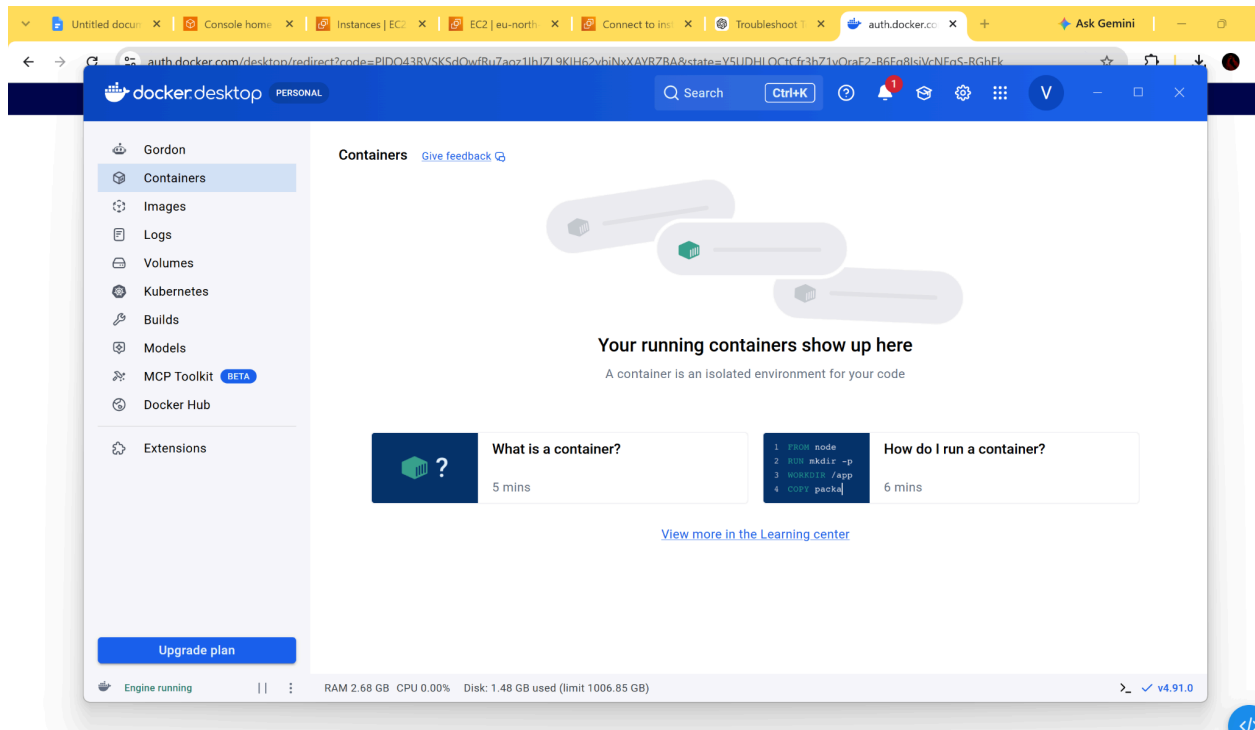
7. **Test Docker**

docker run hello-world

8. Check containers

docker ps

Result: Docker Desktop is successfully installed and running on Windows.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> docker --version
Docker version 29.8.0, build 88096ef
PS C:\WINDOWS\system32> docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

PS C:\WINDOWS\system32>
```